A	0	10000001	01110000000000000000000
B	1	10000001	11100000110000000000000

Denormalization results in:

	S	E	Denormalized M
A	0	10000010	10111000000000000000000
B	1	10000010	11100000110000000000000

Alignment is not needed (both exponents are the same), so we apply addition operation on the combinations of sign and mantissa. The result is shown below, in which the sign of the result is negative:

	S	E	Denormalized M
R	1	10000010	00101000110000000000000

Now we need to normalize. We decrement the exponent three times and shift the denormalized mantissa to the left three positions:

	S	E	M
R	1	01111111	01000110000000000000000

The mantissa is now 24 bits, so we round it to 23 bits.

	S	E	M
R	1	01111111	01000110000000000000000

The result is R = $-2^{127-127}$ X 1.0100011 = -1.2734375, as expected.

4.4 END-CHAPTER MATERIALS

4.4.1 RECOMMENDED READING

For more details about the subjects discussed in this chapter, the following books are recommended:

- Mano, M. *Computer System Architecture,* Upper Saddle River, NJ: Prentice-Hall, 1993.
- Null, L. and Lobur, J. *Computer Organization and Architecture,* Sudbury, MA: Jones and Bartlett, 2003.
- Stalling, W. *Computer Organization and Architecture,* Upper Saddle River, NJ: Prentice-Hall, 2000.

4.4.2 KEY TERMS

This chapter has introduced the following key terms, which are listed here with the pages on which they first occur:

AND operator 79

arithmetic operation 86

arithmetic shift operation 84

Boolean algebra 78

circular shift operation 84

logic operation 78

logical shift operation 83

mask 81

NOT operator 79

OR operator 79

set 82

truth table 78

unset 82

XOR operator 79

4.4.3 SUMMARY

- Operations on data can be divided into three broad categories: logic operations, shift operations, and arithmetic operations. Logic operations refer to those operations that apply the same basic operation to individual bits of a pattern or to two corresponding bits in two patterns. Shift operations move the bits in the pattern. Arithmetic operations involve adding, subtracting, multiplying, and dividing.
- The four logic operators discussed in this chapter (NOT, AND, OR, and XOR) can be used at the bit level or the pattern level. The NOT operator is a unary operator, while the AND, OR, and XOR operators are binary operators.
- The only application of the NOT operator is to complement the whole pattern. One of the applications of the AND operator is to unset (force to 0) specific bits in a bit pattern. One of the applications of the OR operator is to set (force to 1) specific bits in a bit pattern. One of the applications of the XOR operator is to flip (complement] specific bits in a bit pattern.
- Shift operations move the bits in the pattern: they change the positions of the bits. We can divide shift operations into two categories: logical shift operations and arithmetic shift operations. A logical shift operation is applied to a pattern that does not represent a signed number. Arithmetic shift operations assume that the bit pattern is a signed integer in two's complement format.
- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to integers. Integers are normally stored in two's complement format. One of the advantages of two's complement representation is that there is no difference between addition and subtraction. When the subtraction operation is encountered, the computer simply changes it to an addition operation, but forms the two's complement of the second number. Addition and subtraction for integers in sign-and-magnitude representation looks very complex. We have eight situations to consider.
- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to reals stored in floating-point format. Addition and subtraction of real numbers stored in floating-point numbers is reduced to addition and subtraction of two integers stored in sign-and-magnitude after the alignment of decimal points.

4.5 PRACTICE SET

4.5.1 QUIZZES

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

4.5.2 REVIEW QUESTIONS

Q4-1. What is the difference between an arithmetic operation and a logical operation?

Q4-2. What happens to the carry from the leftmost column in the addition of integers in two's complement format?

integers in two's complement format?

Q4-3. Can *n*, the bit allocation, equal 1 ? Why or why not?

Q4-4. Define the term *overflow*.

Q4-5. In the addition of floating-point numbers, how do we adjust the representation of numbers with different exponents?

Q4-6. What is the difference between a unary operation and a binary operation?

Q4-7. Name the logical binary operations.

Q4-8. What is a truth table?

Q4-9. What does the NOT operator do?

Q4-10. When is the result of an AND operator true?

Q4-11. When is the result of an OR operator true?

Q4-12. When is the result of an XOR operator true?

Q4-13. Mention an important property of the AND operator discussed in this chapter.

Q4-14. Mention an important property of the OR operator discussed in this chapter.

Q4-15. Mention an important property of the XOR operator discussed in this chapter.

Q4-16. What binary operation can be used to set bits? What bit pattern should the mask have?

Q4-17. What binary operation can be used to unset bits? What bit pattern should the mask have?

Q4-18. What binary operation can be used to flip bits? What bit pattern should the mask have?

Q4-19. What is the difference between logical and arithmetic shifts?

4.5.3 PROBLEMS

P4-1. Show the result of the following operations:

- **a.** NOT $(99)_{16}$
- **b.** NOT $(FF)_{16}$
- **c.** NOT $(00)_{16}$
- **d.** NOT$[01)_{16}$

P4-2. Show the result of the following operations:

- **a.** $(99)_{16}$AND $(99)_{16}$
- **b.** $(99)_{16}$AND$(00)_{16}$
- **c.** $(99)_{16}$AND$(FF)_{16}$
- **d.** $(FF)_{16}$AND$(FF)_{16}$

P4-3. Show the result of the following operations:

- **a.** $(99)_{16}$ OR $(99)_{16}$
- **b.** $(99)_{16}$ OR $[00)_{16}$
- **c.** $(99)_{16}$ OR $(FF)_{16}$
- **d.** $(FF)_{16}$ OR $(FF)_{16}$

P4-4. Show the result of the following operations:

- **a.** NOT$[(99)_{16}$OR$[99)_{16})]$
- **b.** $(99)_{16}$ OR [NOT $[00)_{16}$]

c. $[(99)_{16}$AND $(33)_{16})]$ OR $[(00)_{16}$AND $[FF)_{16}]$

d. $(99)_{16}$ OR $(33)_{16}$AND $[[00)_{16}$ OR $[FF)_{16}]$

P4-5. We need to unset (force to 0) the four leftmost bits of a pattern. Show the mask and the operation.

P4-6. We need to set (force to 1) the four rightmost bits of a pattern. Show the mask and the operation.

P4-7. We need to flip the three rightmost and the two leftmost bits of a pattern. Show the mask and the operation.

P4-8. We need to unset the three leftmost bits and set the two rightmost bits of a pattern. Show the masks and operations.

P4-9. Use the shift operation to divide an integer by 4.

P4-10. Use the shift operation to multiply an integer by 8.

P4-11. Use a combination of logical and shift operations to extract the fourth and fifth bits from the left of an unsigned integer.

P4-12. Using an 8-bit allocation, first convert each of the following integers to two's complement, do the operation, and then convert the result to decimal.

a. 19 + 23

b. 19-23

c. -19 + 23

d. -19-23

P4-13. Using a 16-bit allocation, first convert each of the following numbers to two's complement, do the operation, and then convert the result to decimal.

a. 161 + 1023

b. 161 -1023

c. -161 + 1023

d. -161 - 1023

P4-14. Which of the following operations creates an overflow if the numbers and the result are represented in 8-bit two's complement representation?

a. 11000010 + 00111111

b. 00000010 + 00111111

c. 11000010+ 11111111

d. 00000010+ 11111111

P4-15. Without actually doing the calculation, can we tell which of the following creates an overflow if the numbers and the result are in 8-bit two's complement representation?

a. 32 + 105

b. 32-105

c. -32 + 105

d. -32- 105

P4-16. Show the result of the following operations assuming that the numbers are stored in 16-bit two's complement representation. Show the result in hexadecimal notation.

a. $(012A)_{16}+(0E27)_{16}$

b. $(712A)_{16}+(9E00)_{16}$
c. $[8011)_{16}+[0001)_{16}$
d. $(E12A)_{16}+(9E27)_{16}$

P4-17. Using an 8-bit allocation, first convert each of the following numbers to sign-and-magnitude representation, do the operation, and then convert the result to decimal.

a. 19 + 23
b. 19-23
c. -19 + 23
d. -19-23

P4-18. Show the result of the following floating-point operations using IEEE_127—see Chapter 3.

a. 34.75 + 23.125
b. -12.625 + 451.00
c. 33.1875-0.4375
d. -344.3125-123.5625

P4-19. In which of the following situations does an overflow never occur? Justify the answer.

a. Adding two positive integers.
b. Adding one positive integer to a negative integer.
c. Subtracting one positive integer from a negative integer.
d. Subtracting two negative integers.

P4-20. What is the result of adding an integer to its one's complement?
P4-21. What is the result of adding an integer to its two's complement?